ADA Lab Assignment

Roll no. – 2401730230

Name – Bishnu kumar

Class – Ai & Ml section C

Q1 Fractional knapsack

Code-

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Item {
    double value, weight;
};

// Comparator: sort by value/weight ratio descending
bool cmp(Item a, Item b) {
    return (a.value / a.weight) > (b.value / b.weight);
}

double fractionalKnapsack(double capacity, vector<Item>& items) {
    sort(items.begin(), items.end(), cmp);

    double totalValue = 0.0;

    for (auto& item : items) {
        if (capacity == 0) break;

        if (item.weight <= capacity) {
            // Take the whole item
            totalValue += item.value;
            capacity   -= item.weight;
        } else {
            // Take a fraction
            double fraction = capacity / item.weight;
            totalValue += item.value * fraction;
            capacity = 0;
        }
    }
    return totalValue;
}
```

```
int main() {
    double capacity = 50;
    vector<Item> items = {{60,10}, {100,20}, {120,30}};

    cout << "Max value: " << fractionalKnapsack(capacity, items) << endl;
    // Output: Max value: 240
    cout <<"name - bhisnu kumar , roll no - 2401730230"<<endl;
    return 0;
}
```

Output

Q2 Matrix Chain Multiplication

Code-

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

void printOptimalParens(vector<vector<int>>& bracket, int i, int j) {
    if (i == j) { cout << "M" << i+1; return; }
    cout << "(";
    printOptimalParens(bracket, i, bracket[i][j]);
    printOptimalParens(bracket, bracket[i][j]+1, j);
    cout << ")";
}

int matrixChainOrder(vector<int>& dims) {
    int n = dims.size() - 1;   // number of matrices
    vector<vector<int>> dp(n, vector<int>(n, 0));
    vector<vector<int>> bracket(n, vector<int>(n, 0));

    // chain_len = length of chain being considered
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
```

```
                int cost = dp[i][k] + dp[k+1][j]
                         + dims[i] * dims[k+1] * dims[j+1];
                if (cost < dp[i][j]) {
                    dp[i][j]      = cost;
                    bracket[i][j] = k;
                }
            }
        }
    }

    cout << "Optimal parenthesization: ";
    printOptimalParens(bracket, 0, n-1);
    cout << endl;

    return dp[0][n-1];
}

int main() {
    // Matrices: 10x30, 30x5, 5x60, 60x10
    vector<int> dims = {10, 30, 5, 60, 10};
    cout << "Min multiplications: " << matrixChainOrder(dims) << endl;
    // Output: Min multiplications: 27000
    cout <<"name - bhisnu kumar , roll no - 2401730230"<<endl;
    return 0;
}
```

Output-

Q3 Longest Common Subsequence(LCS)

Code-

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

pair<int, string> lcs(const string& s1, const string& s2) {
    int m = s1.size(), n = s2.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));

    for (int i = 1; i <= m; i++) {
```

```
        for (int j = 1; j <= n; j++) {
            if (s1[i-1] == s2[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }

    string result = "";
    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (s1[i-1] == s2[j-1]) {
            result += s1[i-1];
            i--; j--;
        } else if (dp[i-1][j] > dp[i][j-1]) {
            i--;
        } else {
            j--;
        }
    }

    reverse(result.begin(), result.end());
    return make_pair(dp[m][n], result);
}

int main() {
    string s1 = "ABCBDAB";
    string s2 = "BDCAB";

    pair<int, string> result = lcs(s1, s2);
    cout << "LCS Length   : " << result.first  << endl;
    cout << "LCS Sequence : " << result.second << endl;
    // Output: LCS Length   : 4
    //         LCS Sequence : BCAB
    cout<< "name - bhisnu kumar , roll no - 2401730230"<<endl;
    return 0;
}
```

Output-

Q4 0/1 knapsack

Code-

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

pair<int, vector<int>> knapsack01(int capacity,
                                  vector<int>& weights,
                                  vector<int>& values) {
    int n = weights.size();
    vector<vector<int>> dp(n+1, vector<int>(capacity+1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i][w] = dp[i-1][w];
            if (weights[i-1] <= w) {
                int take = dp[i-1][w - weights[i-1]] + values[i-1];
                dp[i][w] = max(dp[i][w], take);
            }
        }
    }

    vector<int> selected;
    int w = capacity;
    for (int i = n; i > 0; i--) {
        if (dp[i][w] != dp[i-1][w]) {
            selected.push_back(i-1);
            w -= weights[i-1];
        }
    }
    reverse(selected.begin(), selected.end());
    return make_pair(dp[n][capacity], selected);
}

int main() {
    int capacity = 50;
    vector<int> weights = {10, 20, 30};
    vector<int> values  = {60, 100, 120};

    pair<int, vector<int>> res = knapsack01(capacity, weights, values);
    int maxVal = res.first;
    vector<int> items = res.second;

    cout << "Max value: " << maxVal << endl;
    cout << "Items (0-based): ";
```

```
    for (int i : items) cout << i << " ";
    cout << endl;
    // Output: Max value: 220
    //         Items: 1 2
    cout <<"name - bhisnu kumar , roll no - 2401730230"<<endl;
    return 0;
}
```

Output-

Q5 Dijksatras Algorithm

Code-

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
using namespace std;

typedef pair<int,int> pii;

vector<int> dijkstra(int src, int V, vector<vector<pii>>& graph) {
    vector<int> dist(V, INT_MAX);
    dist[src] = 0;

    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push(make_pair(0, src));

    while (!pq.empty()) {
        int d = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if (d > dist[u]) continue;

        for (int i = 0; i < graph[u].size(); i++) {
            int v = graph[u][i].first;
            int w = graph[u][i].second;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push(make_pair(dist[v], v));
            }
        }
```

```
    }
    return dist;
}

int main() {
    int V = 5;
    vector<vector<pii>> graph(V);
    graph[0].push_back(make_pair(1, 4));
    graph[0].push_back(make_pair(2, 2));
    graph[1].push_back(make_pair(3, 3));
    graph[1].push_back(make_pair(2, 5));
    graph[2].push_back(make_pair(1, 1));
    graph[2].push_back(make_pair(3, 8));
    graph[2].push_back(make_pair(4, 10));
    graph[3].push_back(make_pair(4, 2));

    vector<int> dist = dijkstra(0, V, graph);

    cout << "Shortest distances from node 0:" << endl;
    for (int i = 0; i < V; i++)
        cout << "  to " << i << " = " << dist[i] << endl;
    // Output: 0->0=0, 0->1=3, 0->2=2, 0->3=6, 0->4=8
    cout <<"name - bhisnu kumar , roll no - 2401730230"<<endl;
    return 0;
}
```

Output-

Q6 Bellman-Forman Algorithm

Code-

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

struct Edge {
    int u, v, w;
};

vector<int> bellmanFord(int src, int V, vector<Edge>& edges) {
```

```
    vector<int> dist(V, INT_MAX);
    dist[src] = 0;

    // Relax all edges V-1 times
    for (int i = 0; i < V - 1; i++) {
        bool updated = false;
        for (int j = 0; j < edges.size(); j++) {
            int u = edges[j].u;
            int v = edges[j].v;
            int w = edges[j].w;
            if (dist[u] != INT_MAX && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                updated = true;
            }
        }
        if (!updated) break;
    }

    // Detect negative-weight cycles
    for (int j = 0; j < edges.size(); j++) {
        int u = edges[j].u;
        int v = edges[j].v;
        int w = edges[j].w;
        if (dist[u] != INT_MAX && dist[u] + w < dist[v]) {
            cout << "Negative cycle detected!" << endl;
            return {};
        }
    }

    return dist;
}

int main() {
    int V = 5;
    vector<Edge> edges;

    edges.push_back({0, 1,  6});
    edges.push_back({0, 3,  7});
    edges.push_back({1, 2,  5});
    edges.push_back({1, 3,  8});
    edges.push_back({1, 4, -4});
    edges.push_back({2, 1, -2});
    edges.push_back({3, 2, -3});
    edges.push_back({3, 4,  9});
    edges.push_back({4, 0,  2});
    edges.push_back({4, 2,  7});

    vector<int> dist = bellmanFord(0, V, edges);
```

```
    cout << "Shortest distances from node 0:" << endl;
    for (int i = 0; i < V; i++)
        cout << "  to " << i << " = " << dist[i] << endl;
    // Output: to 0=0, to 1=2, to 2=4, to 3=7, to 4=-2
    cout <<"name - bhisnu kumar , roll no - 2401730230"<<endl;
    return 0;
}
```

Output-